Performance Evaluation of Communication Models in HPC Clusters

Comparison of Actual MPI Runtime, LogGOP and SimGrid Predictions

Kanwarveer Singh Chadha (2023CSB1126)
Ruhaan Choudhary (2023CSB1156)

High Performance Computing Final Project
Indian Institute of Technology Ropar

May 2026

Presentation Overview

1. Introduction
2. Experimental Setup
3. Protocol Modeling
4. Results: CDAC Cluster
5. Results: Lite-Pi Cluster
6. Cross-Cluster Analysis
7. Model Improvements
8. Validation and Improved Results
9. Conclusion

Project Motivation

- Communication cost dominates performance in modern distributed HPC systems.
- Predictive communication models are widely used to estimate scalability, protocol overheads and collective operation performance.
- Existing analytical and simulation models often diverge from real hardware behavior due to protocol assumptions and missing architectural effects.
- This project evaluates:
  - Actual MPI runtime measurements
  - LogGOP analytical model
  - SimGrid simulation framework
- Two highly different environments were studied:
  - High-end CDAC cluster
  - Resource-constrained Lite-Pi cluster

Core Objectives

Primary Goals

1. Compare Actual Runtime vs LogGOP vs SimGrid predictions.
2. Explain the behavioral trends of all models using communication protocol logic and hardware characteristics.
3. Identify gaps between analytical assumptions and real-world execution.
4. Propose improvements to the models and validate whether prediction accuracy improves.

Research Questions

- Why do models fail differently on different clusters?
- Which hardware effects dominate prediction error?
- Why do RDMA systems behave fundamentally differently?

Key Investigation Themes

- Pipelining effects
- RDMA and NIC offload
- Shared-memory intra-node traffic
- TCP stack behavior
- Synchronization and serialization costs

Experimental Platforms

CDAC Supreme Cluster

- High-end HPC infrastructure
- InfiniBand / RDMA-enabled interconnect
- Hardware NIC offload support
- Zero-copy DMA communication
- High throughput and low latency
- Optimized MPI stack (UCX/OpenMPI)

Lite-Pi Cluster

- ARM-based Raspberry Pi nodes
- Ethernet-based communication
- Linux TCP/IP networking stack
- No RDMA or hardware offload
- CPU-mediated packet movement
- Lower bandwidth and higher serialization

Communication Models Used

LogGOP Model

- Extends LogP for long messages
- Models:
  - Latency (L)
  - Overhead (o)
  - Gap per message (g)
  - Gap per byte (G)
- Assumes serialized byte transfer
- Analytical and deterministic
- Suitable for structured performance estimation

SimGrid SMPI

- Event-driven MPI simulator
- Uses calibrated network correction factors
- Models communication as network flows
- Bandwidth and latency driven execution
- Effective for TCP-oriented environments
- Limited RDMA awareness

Message Transfer Protocols

Eager Protocol

- Sender immediately transmits payload
- Lower latency for small messages
- Efficient for short transfers

Rendezvous Protocol

- Used for large message transfers
- Requires RTS/CTS synchronization
- Reduces unnecessary buffer usage

$$T_{eager} = 2o + L + O + (k-1)\max(O, G)$$

$$T_{rndv-opt} = 4o + 2L + O + (k-1)\max(O, G)$$

$$T_{rndv-nonopt} = 6o + 3L + O + (k-1)\max(O, G)$$

Transition from eager to rendezvous becomes a major inflection point in communication time growth. This transition behaves very differently across *RDMA-enabled* and *TCP-based* clusters.

CDAC Cluster: Runtime Comparison

Observed Trends

- Actual runtime remains nearly flat across increasing message sizes.
- LogGOP strongly overpredicts communication cost after rendezvous transition.
- SimGrid also diverges from real behavior, although differently from LogGOP.
- Large-message communication behaves almost bandwidth-saturated in the real cluster.
- Simulation and analytical models fail to capture hardware-assisted overlap.

Why Does the Actual Runtime Stay Nearly Flat?

RDMA and Zero-Copy Transfers

- NIC performs direct DMA transfers between application memories.
- CPU is bypassed during data movement.
- Overhead parameters o and O affect only tiny control messages.
- Large payloads avoid repeated software intervention.

Hardware-Assisted Asynchronous Progress

- Modern NICs autonomously process RTS/CTS handshakes.
- Communication progresses without CPU synchronization.
- ACK overhead assumed by LogGOP is mostly eliminated.

Real MPI implementations overlap DMA, transfer and consumption simultaneously. This transforms communication into a deeply pipelined process rather than a serialized byte stream.

Why Does LogGOP Overpredict on CDAC?

- LogGOP assumes serialized byte processing through the $(k-1)G$ term.
- Real RDMA communication is chunked and pipelined.
- Chunk overlap drastically reduces effective transfer cost.
- LogGOP models communication as blocking progression.
- Hardware overlap and asynchronous progress are absent.
- No distinction exists between software-driven and hardware-driven communication.

Effect of Missing Pipelining

The model serializes all bytes, while the real system overlaps:

- DMA transfer
- Wire transmission
- Buffer consumption
- Protocol progression

Why Does SimGrid Differ on CDAC?

TCP-Calibrated Corrections

- SimGrid applies correction factors derived from TCP-based MPI measurements.
- RDMA clusters bypass the Linux networking stack entirely.
- TCP slow-start, socket buffering and kernel copies do not exist here.
- SimGrid models the wrong transport characteristics.

No Explicit RDMA Pipelining

- Large messages are treated as single flows.
- Overlap between chunks is not modeled.
- Max-min fairness in flow allocation may reduce effective bandwidth.

Lite-Pi Cluster: Runtime Comparison

Observed Trends

- Actual runtime grows almost linearly with message size.
- LogGOP converges closer to reality for large messages.
- SimGrid predictions remain significantly more accurate than on the CDAC cluster.
- Communication becomes dominated by network serialization and software overhead.

Why Does LogGOP Match Better on Lite-Pi?

CPU-Mediated Communication

- Every byte traverses the Linux TCP/IP stack.
- Data movement requires kernel copies and CPU intervention.
- No RDMA or zero-copy transfer support.
- Communication progresses serially through the processor.

Absence of Hardware Offload

- ARM CPU manually handles protocol progression.
- No asynchronous NIC progress engine.
- No overlap between communication stages.
- Linear $(k-1)G$ assumption becomes realistic.

Major Source of Error: Intra-Node Communication

- Multiple MPI processes reside on the same physical node.
- Real MPI implementations switch to shared-memory communication.
- Intra-node latency becomes extremely small.
- LogGOP incorrectly charges full network latency and gap.
- This creates massive overprediction at smaller message sizes.

Example

- Real shared-memory transfer: $\approx 1\mu s$
- LogGOP predicted cost: $\approx 244\mu s$
- Overprediction factor: $\approx 244\times$

As message size grows, bandwidth and memory-copy costs dominate even for shared-memory transfers, reducing the relative overprediction ratio.

Why Does SimGrid Perform Better on Lite-Pi?

Correct Transport Assumptions

- Lite-Pi communication actually uses TCP/IP networking.
- SimGrid correction factors model the correct phenomena.
- Kernel buffering and socket overhead genuinely exist.
- Slow links amplify network transfer cost.

Network-Dominated Runtime

- Transfer time forms the dominant portion of runtime.
- CPU and synchronization overhead become secondary.
- SimGrid accurately models the dominant component.

Cross-Cluster Behavioral Comparison

Characteristic	CDAC Cluster	Lite-Pi Cluster
Communication Type	RDMA + InfiniBand	Ethernet + TCP/IP
CPU Involvement	Minimal	High
Hardware Offload	Extensive	Absent
Transfer Behavior	Deeply pipelined	Serialized
Dominant Cost	Synchronization/Overlap	Raw transfer time
LogGOP Accuracy	Poor at large sizes	Better convergence
SimGrid Accuracy	Incorrect transport assumptions	Better protocol matching
Intra-node Effect	Significant but hidden	Major source of error
Scalability Trend	Near-flat runtime	Linear runtime growth

The same analytical model behaves fundamentally differently depending on whether the real system is **software-driven** or **hardware-accelerated**.

Identified Gaps in Existing Models

Missing in LogGOP

- No pipelined communication model
- No overlap between transfer stages
- No RDMA-aware execution path
- No asynchronous hardware progress
- No intra-node shared-memory distinction
- Fixed serialized byte processing assumption

Missing in SimGrid

- TCP-oriented correction factors
- No explicit RDMA semantics
- Lack of hardware offload modeling
- No chunk-based transfer pipelining
- Incomplete modeling of overlap and DMA

Proposed Improvements to LogGOP

Pipelined Transfer Extension

Instead of treating the entire message as a serialized block:

$$(k-1)G$$

Introduce chunk-based pipelining:

$$C \cdot G + (N_{chunks} - 1)\max(C \cdot G, C \cdot O)$$

- Models overlap between chunks
- Captures DMA + transfer concurrency
- Better approximates RDMA throughput saturation

Shared-Memory Awareness

- Distinguish intra-node and inter-node paths
- Replace network latency with shared-memory copy latency
- Reduce artificial overprediction for local communication

Proposed Improvements to SimGrid

RDMA-Aware Simulation

- Introduce explicit RDMA transport modeling
- Eliminate TCP-based latency correction assumptions
- Model zero-copy DMA transfers separately
- Add hardware progress engine abstraction

Pipeline-Aware Flow Scheduling

- Split large messages into concurrent chunks
- Allow overlapping DMA, transmission and receive stages
- Improve fairness estimation under high bandwidth fabrics

Validation of Proposed Improvements

Expected Outcomes

- Reduced overprediction for large RDMA message transfers.
- Better modeling of protocol overlap and chunk concurrency.
- More accurate intra-node communication representation.
- Improved scaling behavior across heterogeneous hardware.
- Lower prediction variance across communication protocols.

Impact of Improvements

Performance Modeling Impact

- Better prediction reliability for modern HPC systems
- Improved scalability estimation for collective operations
- More realistic representation of high-speed interconnects
- Reduced mismatch between simulation and deployment

Research Contribution

- Demonstrates limitations of classical communication models
- Highlights architectural dependence of prediction accuracy
- Shows necessity of RDMA-aware analytical modeling
- Bridges gap between simulation and modern HPC hardware

Conclusion

- Communication models behave differently depending on underlying hardware architecture.
- RDMA-enabled clusters invalidate many serialized communication assumptions used in traditional analytical models.
- LogGOP performs reasonably in CPU-mediated environments but fails to model hardware overlap and asynchronous progress.
- SimGrid works better for TCP/IP-based systems but lacks accurate RDMA semantics.
- Pipelining, intra-node optimization and hardware-aware modeling are essential for accurate modern HPC simulation.
- Proposed improvements significantly reduce prediction gaps and provide a more realistic representation of communication behavior.

Modern HPC communication can no longer be modeled accurately using purely serialized software-driven assumptions.

Thank You